

⚙ Settings System: Manager, Controller

SettingsController.

- ❗ Почти все игры, за исключением самых примитивных, требуют наличие настроек. Где-то настройки очень глобальные и детальные, а где-то попроще. Мне достаточно будет пары параметров.



SettingsController.

- ❗ Настройка громкости музыки, громкости эффектов, разрешение экрана, режим окна, сброс к стартовым настройкам, а также ссылка на GITHUB.

В Awake вызывается `InitResolutions()` - инициализация разрешений экрана. Мы получаем все подходящие для данного экрана разрешения, фильтруем их через LINQ, чтобы не было повторений под разную герцовку монитора, оставляя только самую высокую герцовку для каждого разрешения, а также сортируем в порядке убывания. После происходит ещё одна сортировка, где мы учитываем общее количество пикселей. В массив доступных разрешений мы как раз присваиваем полученные результаты. После чего в отдельное поле записываем их как string, и закидываем в DropDown.

Как работает выбор разрешения?



У нас есть `OnResolutionChanged(int index)` - который будет вызываться при выборе колонки в выпадающем списке. Индекс совпадает с индексами в массиве доступных разрешений. Мы просто достаём значение по индексу и устанавливаем его через `Screen.SetResolution`.

К слову, третьим аргументом мы передаём `Screen.fullScreen` - то есть: изменить разрешение, но не изменять текущий режим окна. Если окно в полноэкранном режиме, то рендер графики уменьшится, но физическая картинка останется такой же, растянувшись (картинка станет мыльной). Если же режим в окне, то изменится не только рендер, но и размер окошка игры.

`FullScreenToggle()` - вызывается при нажатии на чекбокс. Меняет спрайт и изменяет режим окна.

`GitHubButton()` - открывает браузером по умолчанию ссылку на гитхаб проекта.

Для ползунков есть `OnMusicVolumeChanged(float value)` & `OnSFXVolumeChanged(float value)` - которые срабатывают при их перетаскивании. Они обновляют текущую громкость через `SettingsService`.

`Show()` & `Hide()` - вызываются либо кнопками, либо из других методов (только `Hide()`).

`DefaultSettings()` - сбрасывает все настройки на стандартные. Заодно и сообщает `SettingsService`, что настройки сбросились.

Игрок может в любое время нажать `alt+enter`, что сменит режим игры на оконный, но эта системная операция никак не уведомит об этом мой скрипт. Есть `OnApplicationFocus(bool)` - но он работает слегка не так, как хотелось бы. Ранее я вызывал `SyncUI()` в методе `Show()`, при открытии меню. Однако при открытом меню это никак не обновляло текущую

галочку FullScreen. В итоге, кто-то меня убедил добавить проверку текущего поля isOnFullScreen и Screen.fullScreen прямо в Update.

ApplySavedSettings() - запишем ссылку на синглтон сервис. Далее в наши поля присваиваем поля из сервиса, обновляем UI. SetValueWithoutNotify, при установке значения в выпадающем списке разрешений, позволит избежать вызова метода OnResolutionChanged(), чтобы не совершать лишних вызовов кода с тяжелой функцией Screen.SetResolution(), которая бы вызывалась повторно, хотя нам это не нужно.

Интересный момент. currentResolutionIndex = Mathf.Clamp(service.ResolutionIndex, 0, availableResolutions.Length - 1); → защита разрешений. Если игрок с 4K монитором скинет сохранение товарищу, у которого монитор 1366x768 - то у второго игрока контролер бы попытался просто присвоить индекс (например 15) из массива длиной 4, возможно захватив мусор из оперативной памяти. А позже попытался бы установить расширение применяя не валидные данные, что вызвало бы ошибки и баги. В нашем случае мы указываем передаваемое поле, минимум - индекс 0, и максимум - доступные разрешения для данного экрана. Если даже сохранения первого окажутся у второго, то будет так: (15, 0, 4) и операция выдаст нам 4, как максимальный возможный результат. Б - Безопасность.

```
1 using System.Collections.Generic;
2 using System.Linq;
3 using TMPPro;
4 using UnityEngine;
5 using UnityEngine.UI;
6
7 public class SettingsMenuController : Manager<SettingsMenuController>
8 {
9     [Header("Audio")]
10    [SerializeField] private Slider musicSlider;
11    [SerializeField] private Slider sfxSlider;
12
13    [Header("Resolution")]
14    [SerializeField] private TMP_Dropdown resolutionDropdown;
15    private Resolution[] availableResolutions;
16    private int currentResolutionIndex = 0;
17
18    [Header("CheckBox")]
19    [SerializeField] private Image checkboxImage;
20    [SerializeField] private Sprite checkedSprite;
21    [SerializeField] private Sprite uncheckedSprite;
22    private bool isOnFullScreen = true;
23
24    [Header("Github")]
25    [SerializeField]
26    private string githubUrl =
27    "https://github.com/ffriendlyggghostt";
28
29    protected override void Awake()
30    {
31        base.Awake();
32
33        Screen.fullScreenMode = FullScreenMode.FullScreenWindow;
34        InitResolutions();
35        gameObject.SetActive(false);
```

```

36     }
37
38     private void Start() => ApplySavedSettings();
39
40     private void Update()
41     {
42         if (Screen.fullScreen != isOnFullScreen)
43         {
44             SyncUI();
45         }
46     }
47
48     public void Hide()
49     {
50         gameObject.SetActive(false);
51         SettingsService.Instance.SaveSettings();
52     }
53
54     public void Show()
55     {
56         gameObject.SetActive(true);
57     }
58
59     public void OnMusicVolumeChanged(float value) =>
60         SettingsService.Instance.SetMusicVolume(value);
61
62     public void OnSFXVolumeChanged(float value) =>
63         SettingsService.Instance.SetSFXVolume(value);
64
65
66     private void InitResolutions()
67     {
68         var allResolutions = Screen.resolutions;
69
70         var filtered = allResolutions
71             .GroupBy(r => new { r.width, r.height })
72             .Select(g => g.OrderByDescending(r => r.refreshRateRatio.v
73             .ToList());
74
75         filtered = filtered
76             .OrderByDescending(r => r.width * r.height)
77             .ToList();
78
79         availableResolutions = filtered.ToArray();
80
81         var options = availableResolutions
82             .Select(r => $"{r.width}x{r.height}")
83             .ToList();
84
85         resolutionDropdown.ClearOptions();
86         resolutionDropdown.AddOptions(options);
87     }
88
89
90     public void OnResolutionChanged(int index)
91     {
92         Resolution res = availableResolutions[index];
93         Screen.SetResolution(
94             res.width,
95             res.height,
96             Screen.fullScreen
97         );
98         currentResolutionIndex = index;
99         SettingsService.Instance.SetResolutionChanged(index);
100     }
101
102     private void SyncUI()

```

```

103     {
104         isOnFullScreen = Screen.fullScreen;
105         SettingsService.Instance.SetIsFullScreen( isOnFullScreen );
106         RefreshFullscreenUI();
107     }
108
109     private void RefreshFullscreenUI()
110     {
111         checkboxImage.sprite =
112             isOnFullScreen ? checkedSprite : uncheckedSprite;
113     }
114
115     public void DefaultSettings()
116     {
117         musicSlider.value = 1f;
118         sfxSlider.value = 1f;
119
120         isOnFullScreen = true;
121         RefreshFullscreenUI();
122         Screen.SetResolution(
123             availableResolutions[0].width,
124             availableResolutions[0].height,
125             true
126         );
127
128         resolutionDropdown.value = 0;
129         currentResolutionIndex = 0;
130         resolutionDropdown.RefreshShownValue();
131
132         SettingsService.Instance.SetDefaultSettings();
133     }
134
135     public void FullScreenToggle()
136     {
137         isOnFullScreen = !isOnFullScreen;
138         RefreshFullscreenUI();
139         Screen.fullScreen = isOnFullScreen;
140         SettingsService.Instance.SetIsFullScreen( isOnFullScreen );
141     }
142
143
144     public void GitHubButton() => Application.OpenURL(githubUrl);
145
146     public void ApplySavedSettings()
147     {
148         var service = SettingsService.Instance;
149
150         musicSlider.value = service.MusicVolume;
151         sfxSlider.value = service.SFXVolume;
152
153         isOnFullScreen = service.IsFullScreen;
154         currentResolutionIndex = service.ResolutionIndex;
155
156         RefreshFullscreenUI();
157
158         currentResolutionIndex = Mathf.Clamp(
159             service.ResolutionIndex,
160             0,
161             availableResolutions.Length - 1
162         );
163
164         Resolution res = availableResolutions[currentResolutionIndex];
165         Screen.SetResolution(res.width, res.height, isOnFullScreen);
166
167         resolutionDropdown.SetValueWithoutNotify(currentResolutionIndex);
168         resolutionDropdown.RefreshShownValue();
169     }

```

SettingsService.

- ❗ Изначально всю информацию о настройках (сами данные) и запрос в SaveManager осуществлял контролер. На самом деле это крайне не верно, если соблюдать архитектурный порядок. UI и данные должны лежать раздельно, более того сама логика - тоже должна лежать отдельно. В моём случае есть UI (который управляется Unity) → есть Controller, который данные не хранит, он реагирует на UI и отправляет информацию в Service. По сути - он является связывающим мостом. Service - содержит все данные настроек, отдаёт их по запросу контролера, и сохраняет через SaveManager - также по запросу контролера.

Сие паттерн называется MVP. Model - View - Presenter. Он отличается от MVC, и в Unity встречается чаще. Данные лежат отдельно, UI лежит отдельно, Presenter - контролер, знает о UI и вручную их обновляет. Поток выглядит вот так: Input → Presenter → Model → Presenter → View.

В MVC View чаще сама наблюдает за Model. А поток выглядит так: Input → Controller → Model changed → View. UI зависит от данных.

InitializeFromSave() - на старте подгружаем данные из SaveManager. Расфасовываем по полям.

SetMusicVolume() \ SetSFXVolume() - меняют данные громкости и вызывают событие о том, что данные изменились. Слушать это событие будет AudioManager. К слову, мы делаем проверку что пришедшее число: $\text{Mathf.Abs}(\text{MusicVolume} - \text{value}) < 0.001f$ (модуль пришедшего числа отличается от предыдущего более чем 0.001, чтобы избежать лишних вызовов). ⇒ **Epsilon comparison.**

SetIsFullScreen() - меняет текущее значение поля fullScreen.

SetResolutionChanged() - меняет текущее значение поля индекса разрешения.

SetDefaultSettings() - сбрасывает настройки к стандартным.

SaveSettings() - присваиваем текущие данные в Data SaveManager'a, и вызываем сохранение.

```

1 using System;
2 using UnityEngine;
3
4 public class SettingsService : Manager<SettingsService>
5 {
6     public float MusicVolume { get; private set; } = 1f;

```

```

7 public float SFXVolume { get; private set; } = 1f;
8 public bool IsFullScreen { get; private set; } = true;
9 public int ResolutionIndex { get; private set; } = 0;
10
11 public event Action<float> OnMusicVolumeChanged;
12 public event Action<float> OnSFXVolumeChanged;
13 public event Action<bool> OnIsFullScreenChanged;
14 public event Action<int> OnResolutionIndexChanged;
15
16 private void Start()
17 {
18     InitializeFromSave();
19 }
20 private void InitializeFromSave()
21 {
22     var data = SaveManager.Instance.Data.settings;
23
24     MusicVolume = data.musicVolume;
25     SFXVolume = data.sfxVolume;
26     IsFullScreen = data.isFullScreen;
27     ResolutionIndex = data.resolutionIndex;
28 }
29
30 public void SetMusicVolume(float value)
31 {
32     if (Mathf.Abs(MusicVolume - value) < 0.001f) return;
33
34     MusicVolume = value;
35     OnMusicVolumeChanged?.Invoke(value);
36 }
37
38 public void SetSFXVolume(float value)
39 {
40     if (Mathf.Abs(SFXVolume - value) < 0.001f) return;
41
42     SFXVolume = value;
43     OnSFXVolumeChanged?.Invoke(value);
44 }
45
46 public void SetIsFullScreen(bool value)
47 {
48     if (IsFullScreen == value) return;
49     IsFullScreen = value;
50     OnIsFullScreenChanged?.Invoke(value);
51 }
52
53 public void SetResolutionChanged(int value)
54 {
55     if (ResolutionIndex == value) return;
56     ResolutionIndex = value;
57     OnResolutionIndexChanged?.Invoke(value);
58 }
59
60 public void SetDefaultSettings()
61 {
62     SetSFXVolume(1f);
63     SetMusicVolume(1f);
64     SetIsFullScreen(true);
65     SetResolutionChanged(0);
66 }
67
68 public void SaveSettings()
69 {
70     var data = SaveManager.Instance.Data.settings;
71
72     data.musicVolume = MusicVolume;
73     data.sfxVolume = SFXVolume;

```

```
74     data.isFullScreen = IsFullScreen;  
75     data.resolutionIndex = ResolutionIndex;  
76  
77     SaveManager.Instance.Save();  
78 }  
79 }
```

Настройки работают корректно, отображаются и изменяются в реальном времени. Всё подгружается при старте игры и всё сохраняется в json документ. Настройки довольно таки простые, но систему можно легко расширить и добавить множество других настроек. Мне этих для данного проекта более чем достаточно.

28.05.2026 23:49 - Не знаю... Кто бы это ни читал... Спасибо, что-ли... 😊
Сложно представить, что кто-то будет возится с чтением документации к какой-то no-name pet-project игре разраба из глубинки. Но если вы это читаете... Пускай Госпожа Удача будет на вашей стороне на этой жизненной тропе. Я всего лишь учусь документировать свою работу и заодно повторяю какие-то аспекты. Пу-пу-пу...